

Ponteiros em C parte 2

prof. João Araujo

Por que precisamos declarar o tipo?

```
int *ptr;
```

```
ptr = &k;
```

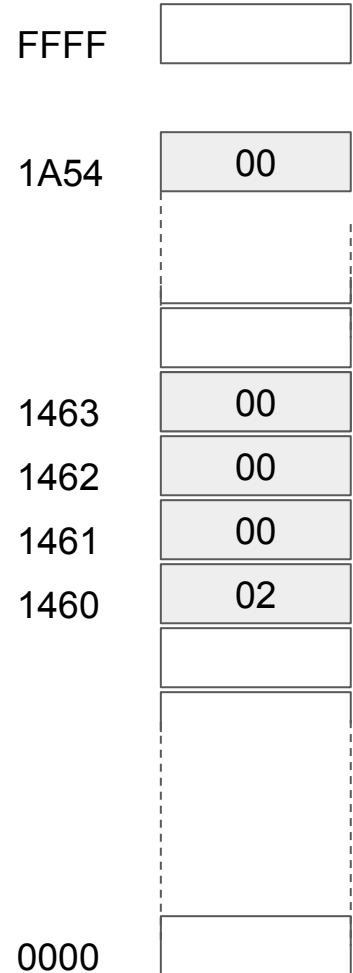
Como ptr “aponta para” algo, se escrevemos:

```
*ptr = 2;
```

o compilador saberá quantos bytes copiar para aquele local de memória apontado por ptr.

Tabela de Símbolos

k	1460
ptr	1A54

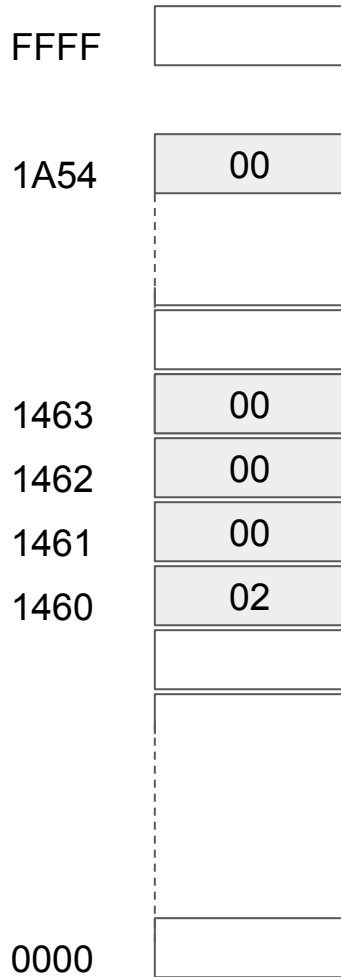


Por que precisamos declarar o tipo?

Se **ptr** for declarado apontando para um inteiro, 4 bytes serão copiados, se for longo, 8 bytes serão copiados. Da mesma forma, para *floats* e *doubles*, o número apropriado será copiado.

Tabela de Símbolos

k	1460
ptr	1A54



Considere um bloco de 10 inteiros

O que acontece quando escrevemos:

```
ptr + 1;
```

Porque o compilador “sabe” que este é um ponteiro (ou seja, seu valor é um endereço) e que aponta para um inteiro (seu endereço atual, **1460**, é o endereço de um inteiro), ele adiciona 4 a ptr em vez de 1, então o ponteiro “aponta para” o próximo inteiro, no local da memória **1464**.

Isto é a “*aritmética de ponteiros*”.

Tabela de Símbolos

k	1460
ptr	1A54

1A54

1460

1484

00000000

1480

00000000

147C

00000000

1478

00000000

1474

00000000

1470

00000000

146C

00000000

1468

00000000

1464

00000000

1460

00000002

Aritmética de ponteiros

Da mesma forma, uma vez que **++ptr** e **ptr++** são ambos equivalentes a

ptr + 1;

Incrementar um ponteiro usando o operador unário ++, seja pré- ou pós-, incrementa o endereço que ele armazena pela quantidade **sizeof(tipo)** onde “tipo” é o tipo do objeto apontado. (ou seja, 4 para um número inteiro, 8 para um longo, etc.).

Tabela de Símbolos

k	1460
ptr	1A54

1A54

1460

1484

00000000

1480

00000000

147C

00000000

1478

00000000

1474

00000000

1470

00000000

146C

00000000

1468

00000000

1464

00000000

1460

00000002

Ponteiros e array

Como um bloco de 10 inteiros localizado contiguamente na memória é, por definição, um array de inteiros, se tivermos a declaração:

```
int meu_array[] = {1, 23, 17, 4, -5, 100};
```

para acessar cada elemento usamos um índice `meu_array[0]` até `meu_array[5]`

Mas podemos também fazê-lo usando um ponteiro:

```
int *ptr;
```

```
ptr = &meu_array[0]; /* o ponteiro aponta para o primeiro  
inteiro em nosso array */
```

Exemplo:

```
#include <stdio.h>
```

```
int meu_array[] = {1, 23, 17, 4, -5, 100};
```

```
#define N sizeof(meu_array) / sizeof(meu_array[0])
```

```
int* ptr;
```

```
int main(void) {
```

```
    ptr = &meu_array[0]; /* aponta nosso ponteiro para o primeiro  
                           elemento do array */
```

```
    for (int i = 0; i < N; i++) {  
        printf("meu_array[%d] = %d\t", i, meu_array[i]); /*<-- A */  
        printf("ptr + %d = %d\n", i, *(ptr + i)); /*<-- B */  
    }
```

```
    return 0;
```

```
}
```

Saída:

`meu_array[0] = 1`

`ptr + 0 = 1`

`meu_array[1] = 23 ptr + 1 = 23`

`meu_array[2] = 17 ptr + 2 = 17`

`meu_array[3] = 4`

`ptr + 3 = 4`

`meu_array[4] = -5 ptr + 4 = -5`

`meu_array[5] = 100 ptr + 5 = 100`

Outros modos

Mude a linha B para:

```
printf("ptr + %d = %d\n", i, *ptr++);
```

e execute-o novamente ... depois mude para:

```
printf("ptr + %d = %d\n", i, *(++ptr));
```

e tente mais uma vez.

&var_nome[0] e var_nome

Em C, o padrão afirma que onde quer que possamos usar **&var_nome[0]**, podemos substituir isso por **var_nome**, portanto, em nosso código, onde escrevemos:

```
ptr = &meu_array[0];
```

podemos escrever:

```
ptr = meu_array;
```

para alcançar o mesmo resultado.

O nome do array é um ponteiro?

O nome do array é o endereço do primeiro elemento no array

Podemos escrever

```
ptr = meu_array;
```

mas não podemos escrever

```
meu_array = ptr;
```

O motivo é que, embora **ptr** seja uma variável, **meu_array** é uma constante. Ou seja, o local em que o primeiro elemento de **meu_array** será armazenado não pode ser alterado depois que **meu_array[]** for declarado.

Memória

Endereço	Memória	Variável
	⋮	
0x5624ea691010	1	meu_array[0]
0x5624ea691014	23	meu_array[1]
0x5624ea691018	17	meu_array[2]
0x5624ea69101c	4	meu_array[3]
0x5624ea691020	-5	meu_array[4]
0x5624ea691024	100	meu_array[5]
0x5624ea691028		
0x5624ea691030	0x5624ea691010	ptr
	⋮	